Document Packet Intelligence & Evidence Retrieval — Technical Report

All figures in this report are measured by the scripts in `scripts/` and written to `outputs/benchmarks/`. Nothing is estimated or copied from published baselines. Where a number could not be measured honestly it is reported as **not measured** rather than filled in.

1. Problem understanding

A packet is a single PDF holding several unrelated documents concatenated together. Three questions must be answered in order, each depending on the previous answer being right:

1. **Where does one document end and the next begin?** A decision over the *N−1 adjacent page pairs*, not over pages — which makes grouping a deterministic consequence of boundary decisions rather than a second model that can disagree with the first.
2. **What is each document, and how sure are we?** Type prediction needs an abstention path: a confident wrong label is worse than an explicit `unknown`.
3. **What is inside it, and where exactly?** Retrieval must cite a document and a page, so structure and page provenance are the substrate the answer is built from, not presentation.

The solution is three stages with dataclass contracts between them (`PageRepresentation → DocumentGroup → Chunk → EvidenceResult`). The contracts are the design: each stage is replaceable without touching its neighbours, which is what let the Stage 3 dense encoder be swapped three times with no change to Stages 1 or 2.

Dataset decision

The brief named DocSplit v2, but that dataset is an evaluation benchmark with no public train/validation split. Following the clarification, **OpenPSS** (`nutrientdocs/openpss-mirror`, SHORT config) is used for boundary detection: Dutch FOIA page streams with per-page `label = 1` marking a document start.

`nutrientdocs/doc-split-benchmark` did prove publicly readable (test split only, as the guidance said). It is therefore **evaluated on but never trained on** — that is what a benchmark is for, and fitting to it is what the brief prohibits. Scoring against it turned out to be the single most informative measurement in this project (§5.1).

OpenPSS carries **no document-type labels**, so it cannot train a type classifier. I added **RVL-CDIP OCR text** (`albertklorer/rvl_cdip_ocr`) — the standard 16-class document-type taxonomy, distributed with OCR words already extracted, which matters because no OCR binary is installed in the target environment. Neither dataset is DocSplit-specific and no benchmark-trained checkpoint is used.

Both are accessed through the HuggingFace `datasets-server` REST API rather than `datasets` + `pyarrow`, because this environment runs Python 3.14 where `pyarrow` has no wheel and fails to build from source without a CMake toolchain.

2. System architecture

Full diagrams: **docs/architecture.md**.

```
PDF → PDFParser (PyMuPDF) → [Stage 1] pairwise features → calibrated GBM → grouping → classifier
                          → [Stage 2] boilerplate → headings → elements/tables → section tree → chunks
                          → [Stage 3] BM25 + dense → RRF → reranker → MMR → evidence + page + confidence
                          → FastAPI /process, /retrieve, /context
```

Stage 1 computes 21 features per adjacent page pair (text delta, visual delta, page-number reset/continuation, header/footer similarity, font and layout deltas), scores them with a calibrated gradient-boosted tree, and splits at the highest-scoring pairs using the expected-count rule (§5.1). Grouping is deterministic from those decisions. Types come from a hybrid classifier described in §3.2.

Stage 2 detects running headers/footers first, then classifies each remaining block as heading, list, caption or paragraph, extracts tables separately, and assembles a section tree with parent links, breadcrumbs and per-element page numbers. Chunks are packed from whole elements.

Stage 3 searches BM25 and a dense index independently, fuses with reciprocal rank fusion, reranks the shortlist on deterministic match features, and normalises a confidence.

3. Design decisions

3.1 Boundary detection as calibrated pairwise classification

Alternatives considered: a sequence model (CRF/BiLSTM) and a similarity-threshold heuristic. Per-pair classification was chosen because it needs no sequence-labelling infrastructure, trains in seconds on CPU, and yields the per-boundary probability the brief asks for. Isotonic calibration was added because raw GBM probabilities on an imbalanced split compress toward zero, making any

fixed threshold unreliable -- and it later proved load-bearing for a second reason, since the calibrated probabilities are what the expected-count decision rule sums (§5.1).

Which gradient-boosting implementation? The learner was initially inherited rather than chosen, so all four common implementations were measured on identical features, calibration, packet split and decision rule (`scripts/compare_boundary_learners.py`):

Learner	Boundary F1	Lift	Page grouping	Train time	Model size
scikit-learn HistGradientBoosting *(shipped)*	0.582	+0.163	0.825	9.6 s	0.8 MB
XGBoost	0.581	+0.163	0.829	1.5 s	0.8 MB
LightGBM	0.586	+0.168	0.828	0.7 s	0.8 MB
CatBoost	0.586	+0.168	0.806	2.9 s	0.5 MB

All four land within **0.005 F1 and 0.005 lift** of each other. That is a second, independent confirmation that the Stage 1 ceiling is representational rather than a property of the learner -- swapping the model family changes nothing, exactly as swapping features and calibration did not.

scikit-learn is retained because the accuracy differences are inside noise, it is already a dependency (LightGBM, XGBoost and CatBoost each add install surface, which has been a real cost in this environment), and `HistGradientBoostingClassifier` is itself a histogram-based GBM modelled on LightGBM. The one genuine difference is training speed: LightGBM fits 13x faster, which would matter for frequent retraining and does not for a one-off. `xgboost` was consequently removed from `requirements.txt`, where it had been declared without ever being imported.

One deliberately non-learned addition: **a printed page number resetting is a near-conclusive boundary cue and is domain-invariant**, unlike the learned score which reflects whichever corpus the model saw. It therefore overrides a muted learned probability. This single rule is what took the 9-page verification packet from one merged blob to four correct documents.

3.2 A hybrid classifier with explicit abstention

RVL-CDIP has no `passport` or `bank_statement` class, and no public labelled corpus for them was available. Rather than pretend, the classifier is two-tier: the trained model handles its 16 classes, and a weighted lexicon covers types with no training data. Reconciliation treats the trained model's confidence as **not comparable** for an out-of-taxonomy type — for a passport the model must assign probability mass to some unrelated class, so it is confidently wrong by construction. A regression test caught this: a stub model returning `letter @ 0.90` beat a correct `passport @ 0.82` under naive numeric comparison.

Confidence is a normalised distribution gated by absolute evidence, not "fraction of this class's keyword list matched" — the latter cannot express ambiguity. `min_confidence` is enforced, so weak evidence yields `unknown`: text reading only `"Total amount 500"` returns **unknown @ 0.111** rather than **invoice @ 0.2**.

3.3 Structure is layout-driven, with a text-only fallback

Every structural signal — heading size, boilerplate position, table regions — derives from bounding boxes and font sizes, which OCR-only sources lack: the engine originally produced *zero* structure on 2,500 real OpenPSS pages. A text-only path reconstructs pseudo-blocks from blank-line grouping and infers headings from typographic convention. Both paths converge downstream.

3.4 Retrieval: measure the encoder rather than assume it

The dense index originally used a hashed bag-of-words whose slots are `int.from_bytes(token) % dims` — related words land in unrelated slots. It scored **R@1 = 0.086**, and fusing it with BM25 produced **R@1 = 0.257 against BM25-alone's 0.714**: the shipped hybrid was worse than deleting half of it. Encoders were made pluggable and all three measured (§6.3).

4. Experimental methodology

Stage	Evaluation set	Labels from
1 — boundaries	OpenPSS SHORT test, 108 streams / 11,354 page pairs	dataset ground truth
1 — boundaries (target)	DocSplit benchmark `our200`, 200 streams / 694 pairs	dataset ground truth (evaluation only)
1 — classification	RVL-CDIP held-out 20%, 2,222 documents	dataset ground truth
2 — structure	two authored fixtures, 12 pages	authored annotations
2 — coverage	OpenPSS SHORT test, 2,500 pages	none (label-free)
3 — retrieval	35 queries over a 515-chunk corpus	authored judgments

Train/test discipline. Stage 1 boundary training uses 95 OpenPSS streams; evaluation uses the full 108-stream test split, with the decision threshold fitted on 54 tuning streams and scored on 54 held-out streams (split by stream, since pairs within a stream are not independent). The DocSplit benchmark is scored but never trained on. The classifier holds out 20% by seeded shuffle.

Why authored fixtures for Stage 2. No public page-stream dataset ships heading/table/caption annotations. I generate the

fixtures programmatically (`scripts/generate_sample_packet.py`, `scripts/generate_benchmark_report.py`) and emit ground truth alongside the PDF, so the labels are authored rather than guessed. **This measures conformance to constructs I chose, not generalisation** — which is precisely why the label-free OpenPSS coverage run exists beside it.

Retrieval judgments are resolved by answer-string containment rather than pinned chunk ids, so they survive re-chunking. The corpus is padded with real OpenPSS chunks as distractors (515 total, of which 38 come from the fixtures), so retrieval is not a 30-way choice.

A metrics honesty fix. Precision/recall/F1 now return `null` when a category has zero expected and zero predicted instances. The sample packet contains no captions; scoring that `0.0` reported correct behaviour as total failure.

5. Benchmark results

5.1 Stage 1 — boundary detection

Full OpenPSS SHORT test split: **108 streams, 11,354 adjacent pairs, 1,296 true boundaries** (11.4% base rate). Threshold is fitted on 54 tuning streams and scored on 54 held-out streams.

Metric	Value
Precision	0.289
Recall	0.551
F1 (held-out threshold)	**0.379**
F1 (threshold tuned on the scored data)	0.406 — biased, shown for contrast
Inference	96 ms per 8-page packet
Model size	0.5 MB (+ 2.0 MB vectoriser)

Verification packet (4 documents / 9 pages): **4/4 documents split correctly**.

How this number was corrected

The first version of this evaluation was wrong twice: the threshold was selected on the data it was then scored on, and the test set held only 12 streams — splitting it in half gave F1 **0.21** on one half and **0.44** on the other, for the same model. Enlarging to the full split and separating threshold selection from scoring fixes both; the optimism gap collapses from 0.167 to 0.027, and the honest figure lands close to the originally reported 0.377, which was therefore roughly right by luck rather than by method.

Test set	Honest held-out F1	Threshold-on-test F1	Gap
12 streams	0.210	0.377	0.167
108 streams	**0.379**	0.406	0.027

Four attempts to raise F1 — all negative

The training matrix showed **8 of 14 features constant** across all 15,905 rows, including `header_similarity` and `footer_similarity`, so the model effectively learned from six. Four hypotheses were tested against the corrected protocol:

Variant	Live features	Precision	Recall	F1	Lift
Baseline (retained)	6 / 14	0.289	0.551	**0.379**	**+0.191**
Pseudo-blocks reconstructed from OCR text	10 / 14	0.287	0.478	0.359	+0.171
Natural class prior (no `balanced` weighting)	6 / 14	0.301	0.486	0.371	+0.183
+ 7 sentence-flow features	13 / 21	0.311	0.458	0.371	+0.183
Synthetic RVL-CDIP packets *(different corpus)*	18 / 21	0.468	0.578	*0.517*	*+0.031*

The first three moved **along one precision/recall curve** rather than shifting it — every variant traded precision up against recall down and landed on the same F1. The threshold already exposes that trade-off, so recalibration and these feature families cannot help; the limit is the model's ability to separate boundary from non-boundary pairs at all.

The fourth is the instructive one. RVL-CDIP ships word-level bounding boxes, so synthetic packets built from it carry real geometry and lifted live features from 6/14 to 18/21 — apparently producing **F1 0.517 against 0.379**. It is not a gain: at that corpus's 32.1% boundary density a trivial always-boundary classifier already scores 0.486, so the lift is +0.031 against the baseline's +0.191. The cause is the corpus, not the model — **RVL-CDIP rows are independent single-page documents** (5,449 pages, 5,449 distinct filenames), so grouping same-class pages into a "document" fabricates continuity that does not exist. DocSplit avoids this by building on RVL-CDIP-N-MP, the multi-page variant, which is public (`jordyvl/rvl_cdip_n_mp`) and is taken up in §8. `boundary_lift` entered the metric set here.

Three latent defects were fixed while running these: models now record the feature construction they were trained with (`synthetic_blocks`, `class_weight`) and evaluation reads it back; and a saved model scores with the feature list it was **trained** on

rather than whatever `FEATURE_NAMES` currently holds — without which adding the seven features would have silently broken every previously saved model.

Evaluation on the actual DocSplit benchmark, and the decision rule it forced

`nutrientdocs/doc-split-benchmark` proved publicly readable (config `our200`) and, as the guidance stated, ships **a test split only**. It is used strictly for evaluation, never for training.

Scoring the shipped configuration there exposed a failure that boundary F1 alone had hidden. The decision threshold (0.633) was fitted on OpenPSS, where 11% of adjacent pairs are boundaries. DocSplit packets are short — a median of four pages — and **72% of pairs are boundaries**. At that threshold the model recovers **6% of them**:

Decision rule	Boundary F1	Page grouping accuracy	Documents found vs actual
Fixed threshold 0.633 (OpenPSS-fitted)	0.110	0.216	1.2 vs 3.5
Expected-count (shipped)	**0.537**	**0.615**	2.2 vs 3.5
Threshold refitted on DocSplit itself	0.851	0.868	4.2 vs 3.5

A single global threshold cannot serve both regimes, and refitting it on the target corpus would be benchmark-fitting. The shipped rule instead exploits the isotonic calibration already applied to the model: **for calibrated probabilities, their sum over a packet estimates the expected number of boundaries in it.** Splitting at the highest-scoring K pairs, K = round(sum(p)), therefore adapts per packet using nothing but the model's own output — no threshold, no target labels.

That change nearly triples page grouping accuracy on the target regime (0.216 → 0.615) for a cost of 0.012 on the training regime (0.784 → 0.772), and is the configuration that ships (`boundary.decision: expected_count`). Its ceiling is still below a threshold refitted on DocSplit, so the gap is real but no longer catastrophic.

We attempted to derive an adaptive threshold from packet length instead, since short packets might plausibly be denser. OpenPSS does not support it: its 2–6 page streams average 22.8% boundary density against 17.1% for its longest, nowhere near DocSplit's 72%. **The training corpus contains no signal that would let a length prior anticipate the target regime**, which is why the calibration-based rule — which needs no such prior — was chosen.

The underlying cause remains a **regime inversion**:

Corpus	Streams	Median pages/stream	Boundary rate	Rare, informative class
OpenPSS (trained on)	108	21	11.4%	boundary
DocSplit (target)	200	4	72.3%	**continuation**

The model learned that boundaries need strong evidence, because in its training corpus they are rare. In the target corpus the rare and informative event is a continuation. The decision rule mitigates this; only retraining in the target regime removes it (§8).

What finally moved the number: matching the training regime

Every earlier attempt changed the model. The one that worked changed the *data*.

OpenPSS boundary labels delimit real multi-page documents, and 67% of them are 1-3 pages. Cutting those documents out and reassembling them into short packets produces training data with the target's shape — median 4 pages, 58% boundary density against DocSplit's 4 and 72% — while both boundaries and continuations remain genuine. This is what the RVL-CDIP attempt could not do, since its rows are independent single pages.

Model	DocSplit F1	DocSplit grouping	OpenPSS F1	OpenPSS grouping
Trained on long streams	0.537	0.615	0.353	0.772
Trained on short packets *(shipped)*	**0.678**	**0.701**	0.347	0.761
Trained on both, mixed	0.518	0.588	**0.388**	**0.789**
Trivial all-boundary	*0.840*	*0.854*	*0.200*	*0.683*

Regime-matched training is worth **+0.141 F1 and +0.086 grouping** on the target for –0.011 grouping on long streams. Mixing the two corpora does *not* split the difference: at an 8:1 pair ratio the long-stream data dominates and the model reverts to the low-density prior, scoring best on OpenPSS and worst on DocSplit. The regime you train on is the regime you are good at.

The honest ceiling

Measured against the right baseline, the system does not yet beat a one-line heuristic on the target regime:

	Our model	Trivial "every page is a document"
DocSplit page grouping	0.701	**0.854**
DocSplit boundary F1	0.678	**0.840**
OpenPSS page grouping	**0.761**	0.683

OpenPSS boundary F1	**0.347**	0.200

On long streams the model adds real value (+0.078 grouping over trivial). On short packets it does not: at 72% boundary density, splitting everything is a strong strategy, and beating it requires reliably detecting the minority event — a *continuation* — which these features do not capture well enough. Precision 0.80 against a 0.72 base rate is only marginally better than chance.

This is stated rather than presented as a success because the earlier framing of grouping 0.615 as an improvement, without its baseline, was the same mistake this report criticises elsewhere. The remedy is a learned page-pair representation that models continuation semantically (§8), not more hand-built features -- four feature sets and four learners have now failed to move this ceiling.

> **Superseded.** This ceiling held for six attempts and turned out to be language-bound, not fundamental. Every corpus named in this section is Dutch, while the model is deployed on English. Retraining in the deployment language clears the trivial baseline by +0.188. The analysis above is retained because it was the honest reading of the evidence available at the time, and because the reason it was wrong is itself the finding — see *Seventh attempt: the ceiling was a language artefact* below.

Sixth attempt: a cross-encoder over the page seam, and what scaling its data did

Five attempts had failed identically: lexical features cannot represent whether text *continues* across a page break. A cross-encoder attends jointly over both pages and can.

`distilbert-base-multilingual-cased` (135M) was fine-tuned as a binary classifier. Three choices carried it: only the **seam** is fed in (the last ~110 words of page *i*, the first ~110 of page *i+1*), since continuation evidence lives at the break and not in body text; the backbone is **multilingual**, because OpenPSS and DocSplit are both Dutch; and the loss is **class-weighted**, or a boundary-heavy corpus pushes the model toward the trivial always-split behaviour it exists to beat.

It was then trained twice, changing only the amount of data. Fetching page *text without images* made the larger corpus practical — images are ~2 orders of magnitude larger and only the feature model's `visual_delta` needs them — which allowed pooling the full SHORT split with the LONG config into 15,485 documents and 3,105 short packets.

Measured on the **full 200-stream** DocSplit benchmark:

Model	Training pairs	Boundary F1	Page grouping	Lift vs trivial
Feature model (21 features, GBM)	2,002	0.537	0.701	−0.153
Cross-encoder	1,600	0.568	0.636	−0.218
Cross-encoder, 5.3x data	**8,460**	**0.776**	**0.804**	**−0.049**
Trivial "every page is a document"	—	*0.839*	*0.854*	*0.000*

Data volume was the dominant factor. Holding architecture, seam window, density and evaluation fixed, going from 1,600 to 8,460 training pairs moved page grouping from 0.636 to 0.804 and closed roughly two thirds of the gap to the trivial baseline. The earlier model was badly underfitted, not badly designed.

It still does not beat trivial on short packets, by 0.049. That is far closer than anything else tried, and the trend across three points is monotone in data, so more data is the obvious next step rather than another architecture.

A caution about the measurement. An intermediate evaluation on a 40-stream subset reported this model at grouping 0.835 against a trivial 0.775 — a *positive* lift of +0.059, which would have been the project's headline result. It does not survive the full benchmark: the subset's trivial baseline was 0.775 against the full set's 0.854, so the subset was materially easier. Both numbers are honest measurements of different samples, and only the 200-stream one is reported above. The 40-stream figure is recorded here because nearly publishing it was the closest this project came to repeating exactly the error it documents elsewhere.

Cost. Inference runs ~0.27 s per page pair against microseconds for the feature model, and training took 2h15m on CPU at 256-token sequences. The feature model therefore remains the configured default; the cross-encoder ships as a reproducible alternative (`scripts/train_cross_encoder.py`, `scripts/evaluate_cross_encoder.py`, `scripts/fetch_openpss_text.py`, `scripts/build_short_packets.py`).

Seventh attempt: the ceiling was a language artefact

Every number above was measured on Dutch. OpenPSS is Amsterdam municipal correspondence and the DocSplit benchmark is drawn from the same source. The report states this two paragraphs earlier — it is the stated reason the cross-encoder backbone is multilingual — but the consequence for the *feature* model was never followed through.

`text_delta`, the first and most heavily weighted feature, is `1 - cosine(tfidf(left), tfidf(right))`, and the vectoriser is fitted on the training corpus. Fitted on Dutch, its 50,000 terms are Dutch: `haarlem`, `haarlemmerdijk`, `haarlemmersluis`. Scored against an English packet it recognises **27.6%** of tokens, so the model's strongest text signal was computed on three-quarters out-of-vocabulary input.

A second, sharper defect sits in `heuristic_features.PAGE_NUMBER`. The pattern matches `Page 2 of 2` and bare digits, but not `Pagina 2 van 2` — which is literally what the OpenPSS pages say. Both `page_number_reset` and `page_number_continuation` were therefore constant zero across the whole Dutch training set, joining the eight already-dead features documented above.

Retraining the identical architecture on TABME++ (English, 504 packets / 2,269 pairs) and evaluating on its **held-out validation split** — 510 packets, 2,239 seams, 58.2% boundary density, no document shared with training:

Model	Boundary F1	Precision	Recall	Page grouping	Lift vs trivial	Vocab coverage
`boundary_shortpackets` (Dutch)	0.481	0.809	0.342	0.518	−0.255	27.6%
`boundary_tabme` (English)	**0.925**	**0.938**	**0.911**	**0.947**	**+0.188**	**81.0%**
Trivial "every page is a document"	*0.736*	—	—	—	*0.000*	—

This is the first configuration in the project to beat the trivial baseline by a margin that is not arguable (+0.188 at 58% density). The failure mode of the Dutch model on English is visible in its recall: 0.342, against precision 0.809. It splits rarely and is usually right when it does — it silently merges documents rather than over-splitting them.

The direction reverses on Dutch, as it must: on OpenPSS the English model scores 0.606 grouping against the Dutch model's 0.761, and on DocSplit 0.651 against 0.701. Neither model is better in general. **The choice of model is a choice of language**, and the ceiling documented in *The honest ceiling* above is a Dutch ceiling, not a property of the approach.

Caveat on the measurement. The validation split shares a corpus with training, which is the friendliest test available and makes 0.947 an optimistic figure. The corroborating evidence is an independently authored 13-page English packet (`scripts/generate_stress_packet.py`, 7 documents, 50% density, no relation to the training corpus): the Dutch model scores 0.846 grouping there with lift −0.121, the English model 0.936 with lift **+0.133**. Two unrelated English sources agreeing near 0.94 is what makes the result credible rather than memorised.

Why this was missed for so long. Every benchmark in this project was Dutch, so a Dutch-trained model was always evaluated in its own language and never looked broken. It took running the system on an English packet — the language of every document the pipeline was demonstrated on — for the mismatch to surface. Choosing an evaluation set in the deployment language is not a refinement; it is the difference between 0.518 and 0.947.

Eighth attempt: selecting the decision rule under a no-leakage protocol

Every result so far used `expected_count`. Choosing between it and a tuned threshold is a selection problem, and this project has already been burned once by selecting on the data it then reported (§*How this number was corrected*). The protocol was therefore fixed in advance: sweep on validation, commit to page grouping accuracy as the selection metric *before* looking, freeze the parameter, then open the test split exactly once. A third rule was included — `hybrid`, splitting where `p >= ceiling` regardless of rank, or where the seam is in the top K and `p >= floor` — since it addresses both directions in which `expected_count` fails.

Held-out test results, each set opened once, each corpus paired with its language-matched model:

Corpus	Density	expected_count	tuned threshold	hybrid	inherited 0.6334
TABME++ test (English)	56.3%	0.9570	**0.9682**	0.9679	0.9516
OpenPSS test (Dutch)	11.0%	**0.7179**	0.6915	0.6964	0.6684
DocSplit test (Dutch)	69.8%	0.6784	*0.8381*	*0.8414*	0.6370

Only the English arm produced a transferable win. Threshold 0.185011 beat `expected_count` by **+0.0112, 95% bootstrap CI [+0.0040, +0.0192]**, having also led on validation — and it is not degenerate: precision 0.897 against a 0.563 base rate.

The other two arms are the instructive ones.

OpenPSS shows the protocol earning its keep. The tuned threshold won validation by **+0.068** and lost test by **−0.027**, with a CI spanning zero. Selecting and reporting on one set would have published a phantom improvement of exactly the kind this report documents elsewhere.

DocSplit shows the selection metric being gamed. The sweep picked threshold **0.0037** with recall **1.000** — it wins by abandoning the model and splitting everywhere. Grouping accuracy rose 0.678 → 0.838 while lift over trivial stayed at **+0.000**. Any future tuning must report lift alongside grouping, or a sweep will happily select a rule that ignores the model entirely.

The hybrid did not earn its place. It tied the plain threshold on English (0.9679 vs 0.9682) and lost on OpenPSS, adding a second parameter for no measurable gain. Recorded as a negative result.

The threshold is a property of `models/boundary_tabme`'s calibrated score distribution, not a general setting — the inherited 0.6334, fitted for the Dutch model, scores 0.9516 on the same data. Dutch deployments keep `expected_count`, which wins there and needs no tuning.

Ninth attempt: is the count estimate itself sound, and can K be softened?

`expected_count` rests on one statistical claim: for calibrated probabilities, the sum over a packet estimates how many boundaries it contains. That claim is testable (`scripts/analyse_expected_count.py`).

Corpus	ECE	mean Σp	mean true	bias	Pearson r	K exact
TABME++ test (English)	**0.025**	2.450	2.565	−0.115	**0.900**	**76.2%**
OpenPSS (Dutch)	0.136	22.345	12.000	**+10.345**	0.559	15.7%

DocSplit (Dutch)	**0.244**	1.881	2.510	−0.629	0.506	27.0%

The ordering is identical on both columns: **calibration quality is count-estimation quality.** Where the model is calibrated the assumption holds strongly; where it is not, Σp is wrong by ten boundaries per packet. Calibration is fitted on the training distribution and does not transfer to a corpus of different density.

The two failure modes are opposite by language. On English all ten reliability bins are *under*-confident, so Σp under-estimates and K starves real boundaries: **46.7% of missed boundaries were seams the model scored ≥ 0.5**, while only 4 weak seams were forced in across 501 packets, and the median margin between weakest-kept and strongest-dropped is a decisive +0.765. On OpenPSS the top bin predicts 0.949 against an observed 0.293, so Σp inflates K, **40.7% of packets are forced to admit a sub-0.5 seam** (134 of them, 89 wrong), and the median margin is +0.054 — the rule is discriminating between near-identical scores.

Six soft-K variants were then simulated on validation (`scripts/simulate_soft_k.py`), each derived rather than tuned: under calibration the expected gain from cutting a seam is `2p − 1`, positive exactly when `p > 0.5`, which is the Bayes rule rather than a fitted constant.

Variant	TABME++ validation grouping
`expected_count` baseline	0.9472
`ceil(Σp)` instead of `round(Σp)`	**0.9637**
top-K, plus admit any seam `p > 0.5`	0.9538
top-K, minus picks below `p < 0.5`	0.9466
residual-mass top-up	0.9547
threshold 0.185 (reference)	*0.9603*

`ceil` led by **+0.0165** over the baseline — the minimal correction for a measured negative bias, with no new parameter, and ahead of even the tuned threshold on this split. It was pre-registered and taken to test once (`scripts/confirm_ceil_rule.py`). **It did not transfer.**

Rule	Test grouping	vs `round`	vs shipped threshold
`round(Σp)`	0.9570	—	—
`ceil(Σp)`	0.9596	+0.0025, CI [−0.0062, +0.0118]	−0.0087, CI [−0.0137, −0.0042]
threshold 0.185	**0.9682**	—	—

Six sevenths of the validation gain was specific to that split, and `ceil` is *significantly worse* than the rule already shipped. The mechanism worked exactly as designed — starved boundaries fell 42 → 2 — but it converges on threshold behaviour by inflating K, and a threshold expresses that regime more directly than a count budget does.

The residual-mass variant is recorded as actively unsafe: it looks fine on English (+0.0075) and produces **8,633 splits against 2,406** on OpenPSS at precision 0.133, because on 100-seam streams the remaining mass stays above 0.5 almost indefinitely. It has no stopping guarantee proportional to stream length.

The conclusion is not that Σp is a bad estimator. On English it is a good one — r 0.90, exactly right three times in four. It is that being a good count estimator does not justify *enforcing* that count: K is useful information and a harmful constraint. The hybrid sweep reached the same verdict independently by optimising its floor to zero and its ceiling to 0.173, discarding the top-K component altogether.

Summary of Stage 1 boundary detection

Shipped configuration: `models/boundary_tabme` (English, TABME++) with `decision: threshold` at **0.185011**. `models/boundary_shortpackets` with `expected_count` ships alongside for Dutch.

Corpus	Language	Role	Boundary F1	Page grouping	Trivial grouping
TABME++ test (501 packets)	English	held-out, deployment language	**0.942**	**0.968**	0.746
Stress packet (13 pages, 7 docs)	English	independent, hand-authored	0.909	**0.974**	0.803
OpenPSS SHORT test	Dutch	wrong language for this model	0.200	0.606	0.683
DocSplit `our200`	Dutch	wrong language for this model	0.625	0.651	0.854

In the language it was trained for the system beats the trivial baseline by **+0.222**; pointed at another language it falls below it. On the independent stress packet the decision-rule change alone lifted page grouping 0.936 → 0.974 and recovered a one-page memo that `expected_count` had dropped for ranking fifth against K=4 — which in turn raised document-type accuracy on that packet from 0.571 to 0.714, since a document that is never separated can never be classified.

Read grouping accuracy, not boundary F1, on any corpus whose boundary density differs from training: at DocSplit's 72% density a classifier marking every pair a boundary scores F1 0.815, so F1 there is dominated by the base rate. Grouping accuracy

separates the same two configurations by 0.216 against 0.615 and is the honest signal.

5.2 Stage 1 — document classification

RVL-CDIP, split three ways from 27,235 pages: 16,341 train, 5,447 validation, 5,447 test. Variants were compared on validation only, macro-F1 was fixed as the selection metric before any model was fitted, and test was scored once (`scripts/train_layout_classifier.py`).

Variant	Test accuracy	Test macro-F1
word 1-2 grams *(the previous shipped configuration)*	0.8153	0.7932
word + character 3-5 grams *(shipped)*	**0.8388**	**0.8198**
word + char + 21 page-geometry features	0.8421	0.8179

Two changes were tested together and are separable.

Character n-grams are the real gain: +0.0235 accuracy, +0.0266 macro-F1. The corpus is OCR of scanned pages, so a word-level vocabulary fails in exactly the way that matters — `Invoice` misread as `lnvoice` is simply out of vocabulary and the evidence it carried is lost, while most of its character trigrams survive. The two views cover different failure modes, so both are kept.

Data volume contributed independently. The same word-only configuration scores 0.7844 accuracy on 5,449 pages and 0.8153 on 27,235 — roughly +0.031 for nothing but a longer download. This is the third time in this project that more data moved a number further than a better model did.

The geometry features: selected on validation, and they did not transfer

The third variant adds 21 document-shape descriptors (`src/features/document_shape.py`): column count, left-margin regularity, ink density by page third, where the largest text sits. The hypothesis was that an invoice, a memo and a scientific paper differ in shape long before they differ in vocabulary, and that this should help precisely where OCR text is least reliable.

On validation it led macro-F1 by **+0.0155** and was duly selected. On test it **lost** by 0.0019.

McNemar, word+char vs word+char+shape, on 5,447 test documents	
word+char right, shape wrong	153
shape right, word+char wrong	171
$\chi^2 = 0.892$, **p = 0.345**	not significant

The two models are statistically indistinguishable; the apparent validation gain was about eighteen documents of noise. **The geometry features are therefore not shipped.** This is the third validation winner in this project to fail on held-out test, after an OpenPSS boundary threshold (+0.068 → −0.027) and a `ceil(Σp)` decision rule (+0.0165 → +0.0025), and the reason the three-way split was used here at all.

The finding is narrower than "layout does not help". It helps the classes text cannot reach — `handwritten` gains **+0.120 F1** over the word-only baseline, the largest per-class movement anywhere in this table, because handwriting produces garbage OCR and geometry is the only signal left. That is not enough to move the aggregate, and an aggregate is what a shipped model is chosen on.

What still fails

Class	F1	Precision	Train examples
`file_folder`	**0.418**	0.311	174
`scientific_report`	0.701	0.692	~1,700
`presentation`	0.743	0.721	~1,600
…			
`resume`	0.958	0.956	~1,800

`file_folder` is the outlier and is not a text-classification problem at all: in RVL-CDIP those images are photographs of folder tabs carrying almost no text. Precision 0.311 means two of every three `file_folder` predictions are wrong, so it also acts as a sink that steals predictions from real classes. Removing it from the taxonomy would raise macro-F1 by roughly 0.025 on arithmetic alone; it is retained here only because the taxonomy is inherited from RVL-CDIP rather than chosen.

`passport` and `bank_statement` are lexicon-scored and have **no held-out measurement** — no labelled corpus for them was available. This remains the largest unmeasured area in the system.

The model is 7.8 MB against the previous 3.3 MB, because the character view carries its own 30k feature vocabulary. Runtime footprint moves from 5.8 MB to 10.5 MB, still CPU-only.

5.3 Stage 2 — structure

Metric	4-doc packet	Report fixture
Heading P / R / F1	1.00 / 1.00 / 1.00	1.00 / 1.00 / 1.00
Table F1	1.00	1.00
List F1	1.00	1.00
Caption F1	*not applicable*	1.00
Page-reference accuracy	1.00 (6/6)	1.00 (6/6)
Type-field accuracy	1.00 (10/10)	*not applicable*
Throughput	24.6 pages/s	12.0 pages/s

Label-free coverage on **2,500 real OpenPSS pages**, before and after the text-only fallback:

Signal	Before	After
Sections	12	2,575
Elements	0	31,039
Mean chunk tokens	256.6 (fixed window)	141.3 (semantic)
Text retention	—	97.7%
Throughput	—	1,250 pages/s

`single_page_chunk_ratio` moved from 1.00 to 0.655 here — an **improvement**, because the old 1.00 was false precision: every chunk claimed its document's first page regardless of origin.

5.4 Stage 3 — retrieval

35 queries, 515-chunk corpus with OpenPSS distractors.

Configuration	R@1	R@3	R@5	P@1	MRR	nDCG	Mean lat.	p95	Index
bm25_only	0.714	0.829	0.857	0.714	0.774	0.776	45.5 ms	52.0	1.0 s
dense_hashed_only	0.086	0.200	0.229	0.086	0.134	0.152	44.9 ms	50.9	1.1 s
rrf_hashed *(old default)*	**0.257**	0.457	0.743	0.257	0.407	0.477	47.9 ms	57.0	1.1 s
dense_svd_only	0.514	0.800	0.829	0.514	0.645	0.673	57.7 ms	66.2	8.4 s
rrf_svd	0.600	0.800	0.857	0.600	0.713	0.731	69.5 ms	79.4	8.0 s
rrf_svd_query_aware	0.657	0.800	0.857	0.657	0.743	0.753	54.8 ms	65.0	8.9 s
rrf_svd_reranked *(default)*	**0.771**	0.857	0.886	0.771	0.821	0.819	57.9 ms	64.8	7.5 s
dense_bge_only	0.657	0.857	0.886	0.657	0.755	0.769	85.9 ms	93.0	70.1 s
rrf_bge	0.800	0.914	0.971	0.800	0.867	0.874	95.4 ms	109.8	67.2 s
rrf_bge_reranked	**0.886**	0.971	**1.000**	0.886	0.929	0.928	118.0 ms	129.0	67.0 s

Two findings matter more than the best row. First, **the previously shipped default was worse than its own lexical half** (0.257 vs 0.714) — a broken component silently degraded a working one, which no aggregate score would have revealed without ablation. Second, the reranker adds **+0.171 R@1** on SVD for ~0 ms and no dependency, making it the best value in the table.

Default: `svd` + reranker. bge wins by +0.115 R@1 but needs ~1 GB of torch, which required two install attempts and a DLL failure to get working in this environment. A submission that does not install is worth less than 0.115 R@1. The transformer is one config line away and its numbers are recorded above and in `requirements.txt`.

Reranker ablation

Rank fusion knows only *positions* and discards how well a candidate actually matches, so a second pass rescores the shortlist on deterministic features. It is the best value in Stage 3: **+0.171 R@1 and +0.140 MRR for roughly 2 ms** (27.6 ms to 29.6 ms mean query latency), with no model and no dependency. It runs on the top ~20 fused candidates rather than the corpus, which is what keeps it cheap.

Ablating each feature (leave-one-out, and each alone) shows the contributions are far from equal:

Variant	R@1	R@5	MRR	nDCG
No reranker	0.600	0.857	0.689	0.712
All features	**0.771**	**0.914**	**0.829**	**0.831**
without `coverage`	0.771	0.914	0.824	0.828
without `phrase`	0.743	0.914	0.814	0.821

without `exactness`	0.771	0.914	0.829	0.831
without `structure`	0.714	0.914	0.800	0.810
`coverage` alone	0.714	0.914	0.800	0.810
`phrase` alone	0.714	0.914	0.788	0.801
`exactness` alone	0.600	0.686	0.636	0.630
`structure` alone	0.571	0.629	0.593	0.583

Three findings, none of which the aggregate score showed:

1. **`exactness` is inert.** Removing it changes every metric by exactly zero — it never altered a ranking decision. The explanation is redundancy rather than a bad feature: BM25 already ranks an exact identifier match first, so by the time the shortlist is rescored the identifier chunk is already on top. It is **retained rather than deleted**, because the query set contains only ~6 identifier queries and removing a feature on that much evidence would be fitting to 35 queries.
2. **`structure` matters most in combination and is worst alone** — removing it costs the most (–0.057 R@1) while by itself it scores *below* the no-reranker baseline. It is a tie-breaker, not a ranker.
3. **`coverage` and `phrase` are substantially redundant with each other**: either alone reaches 0.714, and dropping `coverage` while keeping `phrase` costs nothing at rank 1.

The reranker is therefore doing real work, but through feature *interaction* rather than any single strong signal — and a leaner three-feature version would likely perform identically.

Retrieval-side RAG concerns

Stage 3 is the retrieval half of a retrieval-augmented generation system. Generation is deliberately absent: the brief states twice that the objective is not a chatbot and not generated answers, so the pipeline stops at the point where a generator would be called. Three retrieval-side concerns that determine whether a generator *could* be trusted are handled explicitly.

Diversity (MMR). Pure relevance ranking can return five near-paraphrases of one passage: recall looks healthy, but the generator sees one fact repeated and any multi-fact question fails. Maximal Marginal Relevance trades relevance against redundancy. Measured:

`mmr_lambda`	R@1	R@5	MRR	nDCG	Distinct documents in top-5	Duplicate chunks reaching context
1.0 (off)	0.771	0.886	0.821	0.819	1.74	7
0.7 (shipped)	0.771	**0.914**	**0.829**	**0.831**	**2.09**	**0**

It costs nothing at rank 1 and improves every other measure, because near-duplicates were crowding the gold chunk out of the top 5 rather than adding information.

Context assembly. `POST /context` returns a single grounded block: deduplicated on token overlap, truncated to a token budget in rank order (so the budget removes the least relevant evidence rather than clipping the most relevant mid-sentence), and annotated with `[n]` markers resolving to document id, page and section breadcrumb.

Grounding. Every citation carries a page reference, so any downstream claim is checkable against the source page. Retrieval-side citation is the only point at which this can be established -- it cannot be reconstructed after generation.

5.5 Resource summary

Resource	Value
Committed model footprint	**10.5 MB** total
Process RSS (full pipeline)	153 MB
Index peak RAM — SVD	112 MB
Index peak RAM — bge	13.6 MB
Stage 1 inference	96 ms / 8-page packet
Stage 2 structuring	12–24 pages/s native, 1,250 pages/s text-only
Stage 3 query latency	58 ms mean, 65 ms p95
Retrieval through API	6–34 ms (cached retriever)

Everything runs CPU-only. No GPU, no external API, no network call at inference.

6. Error analysis

6.1 Boundary detection

Dominant failure is **recall on adjacent short documents**. Two same-template forms concatenated share fonts, margins and

header text, so every layout feature reads "continuation", and a pair of one-page documents offers no continuation cue to reject. On the held-out English set the shipped model runs precision 0.938 against recall 0.911, so it now errs toward merging rather than over-splitting — the worse direction, since an over-split document is recoverable downstream whereas a merged one loses a document entirely.

The limit is structural rather than per-feature. On the hand-authored stress packet a one-page letter followed by a one-page memo is merged: both are formal business prose, so `text_delta` reads 0.596 — *lower* than four seams that sit inside single documents, where a page of fielded data is followed by a page of tabular data. Cosine distance measures topic change, and a document boundary is not a topic change. That is the ceiling of comparing page *i* with page *i+1* in isolation, and is what §8 item 6 addresses.

6.2 Classification

`passport`/`bank_statement` rest on lexicon evidence with no held-out measurement. A document using unusual vocabulary for those types would fall below `min_confidence` and return `unknown` — the intended degradation, but still a miss.

6.3 Retrieval

From `outputs/error_analysis/stage3_errors.json`, SVD leaves 8 of 35 queries imperfect, bge 4. The pattern is consistent — **low lexical overlap paraphrases**:

Query	SVD	bge	Cause
"How can I contact the applicant by email?"	miss	rank 5	target says `Contact: arjun.mehta@...`; the word "email" never appears
"What programming languages does the candidate know?"	miss	hit	target lists `Python, SQL, C++` without the word "languages"
"What is the total amount due on the invoice?"	rank 2	rank 2	competing Subtotal/GST chunk
"What is the passport number?"	rank 4	rank 2	identifier appears in several documents

False positives are instructive: for the email query SVD returned a Dutch OpenPSS distractor — LSA fitted on a mixed-language corpus produced a latent dimension grouping unrelated administrative text. That is the concrete cost of a corpus-fitted encoder versus a pretrained one.

6.4 Resolved: removed dead code

`src/stage2/llm_fallback.py` defined an LLM fallback contract nothing called. Deleted rather than wired up: the parser exposes no structure-confidence signal to trigger it, and PDF structure is deterministic enough that an LLM would add cost, latency and nondeterminism without addressing any measured failure.

7. Trade-offs consciously made

Trade-off	Decision	Why
Accuracy vs reproducibility	SVD default over bge (–0.115 R@1)	torch install failed twice here; a submission that won't install is worth less
Recall vs precision on boundaries	favour recall	over-split is recoverable, merged documents are not
Simplicity vs coverage	rule-based structure, no LLM	PDF structure is deterministic; an LLM adds cost, latency and nondeterminism for no gain
Model size vs accuracy	20k TF-IDF features	measured *better* accuracy at 1/10th size
Confidence honesty vs looking good	real probabilities + abstention	invoice confidence dropped 0.8 → 0.49, but 0.8 was meaningless

8. Future improvements (in priority order)

1. **Done — retrain in the target regime, where "regime" turned out to include language.** §5.1 (seventh attempt) shows the shipped model had been trained on Dutch and deployed on English, scoring 27.6% of its tokens in vocabulary. Retraining on TABME++ moved held-out English page grouping from 0.518 to 0.947 and cleared the trivial baseline by +0.188. The transferable lesson — evaluate in the deployment language, or the benchmark measures nothing — outranks every model change attempted before it.
2. **Done — select the decision rule properly.** §5.1 (eighth attempt) swept threshold, `expected_count` and a hybrid on validation, froze the choice, and evaluated once on held-out test. English deployment moved to `decision: threshold` at 0.185011 (+0.0112 grouping, CI [+0.0040, +0.0192]); Dutch keeps `expected_count`, which wins there. The hybrid was a negative result. What remains open is the *reason* a threshold helps: `expected_count` cannot express "I found four boundaries I trust and one I do not", and a fixed threshold cannot adapt to packet density. A rule that does both — top-K with an abstention floor and a

confidence override — was tried here and tied; a per-packet density estimate rather than a global parameter is the untried version.

3. **Fix `PAGE_NUMBER` for non-English page furniture.** The pattern matches `Page 2 of 2` but not `Pagina 2 van 2`, which is what the Dutch corpus actually prints, so `page_number_reset` and `page_number_continuation` were constant zero throughout that training run. A one-line change that silently cost two of twenty-one features.
4. **A labelled corpus for `passport`/`bank_statement`.** The largest unmeasured area; every claim about those classes currently rests on a 4-document fixture.
5. **Scale the cross-encoder's training data.** §5.1 shows the cross-encoder is the first change to improve both regimes, trained on only 1,600 examples. OpenPSS has ~90,000 rows unused. This is now a data problem rather than an architecture one. Batching or a smaller backbone would also address its ~0.27 s per page pair, which is three orders of magnitude slower than the feature model.
6. **Sequence decoding, still untried.** Each pair is decided in isolation before `expected_count` picks the top K globally. A Viterbi pass with a document-length prior would use the fact that boundaries do not cluster. Two further directions, lower priority: * **Sequence modelling.** Every current feature compares page *i* with page *i+1* in isolation, while document boundaries are a sequence-labelling problem — documents have length priors and boundaries do not cluster. Adding neighbouring-pair context and decoding with a length prior is the standard remedy and is untried here. * **Learned page-pair representations.** `text_delta` is TF-IDF cosine, which fires on shared vocabulary; consecutive pages of one report and two different reports on one topic look alike to it. A cross-encoder over page-pair text would model *continuation* rather than *overlap*. torch and sentence-transformers are already installed for Stage 3, so this is reachable.
7. **Transformer embeddings as default**, once dependency installation is reliable — worth +0.115 R@1 and a perfect R@5 on the current evaluation set.
8. **Grow the retrieval evaluation set.** 35 queries gives ±0.07 confidence intervals; conclusions about ~0.05 differences are not statistically safe.
9. **Persist the SVD index** rather than refitting per corpus (8 s today).
10. **Recover the visual signal.** `visual_delta` is a 16-bin grey histogram over 224×224 thumbnails and was constant on the RVL-CDIP packets — a downsampled ink-density grid would capture layout rather than tone.

Technical questionnaire

1. Problem understanding. See §1. Boundaries are decisions over N–1 page pairs; grouping is deterministic from them; structure and page provenance are the substrate retrieval cites.

2. Three most important decisions. (a) Calibrated pairwise boundary classification with a domain-invariant page-number-reset override — chosen over a sequence model for CPU cost and per-boundary probabilities. (b) Hybrid classifier with explicit abstention over a single trained model, because no training data exists for two required classes and a confident wrong label is worse than `unknown`. (c) Pluggable retrieval encoders measured by ablation, which is the only reason the harmful default (§5.4) was found.

3. Technology selection. PyMuPDF (fonts + bboxes), pdfplumber (dual ruled/borderless table strategies), scikit-learn (HistGradientBoosting, TF-IDF, LogisticRegression, TruncatedSVD — all CPU-only), BM25 implemented directly, FastAPI. Rejected: `datasets`/`pyarrow` (no Python 3.14 wheel), Camelot (heavier than pdfplumber for equivalent output), sentence-transformers (measured better but ~1 GB — kept as a documented option).

4. Evaluation methodology. See §4. Held-out splits where dataset labels exist; authored fixtures where no public annotations exist, clearly separated from label-free coverage on real data; retrieval judgments anchored to answer strings; ablations rather than single aggregate scores.

5. Failure analysis. See §6. The biggest limitation by far: on the DocSplit benchmark the system scores F1 0.823 against a trivial baseline of 0.815 — a lift of +0.008, i.e. no better than marking every pair a boundary. It was trained on long streams where boundaries are rare (11%) and the target has them common (72%), inverting which class is informative. Secondary limitations: `passport`/`bank_statement` have no held-out measurement, and the SVD encoder misses low-overlap paraphrases.

6. Resource & performance. See §5.5 — 10.5 MB of models, 153 MB RSS, CPU-only. To shrink further: drop SVD for BM25 + reranker (1 s index, 9 MB peak). To scale up: the dense scan is O(N) per query, fine at 515 chunks and not at 10^6 — it needs an ANN index.

7. Trade-offs. See §7.

8. Two more weeks. §8 item 1 first and alone if necessary: rebuild training packets from `jordyvl/rvl_cdip_n_mp` so the model is trained in the target regime, since every other Stage 1 improvement is worth less than closing a +0.008 lift. Then labelling the extension classes, and sequence modelling over page pairs.

9. AI usage declaration.

I used Claude (via Claude Code) as an implementation and debugging tool throughout this project. It wrote the majority of the code under my direction. I set the agenda for what was built and in what order, reviewed its output, and required every claim to be backed by a measurement before accepting it. Several of the most consequential results in this report came from that review rather

than from the tool's initial output:

- **The decision rule in §5.1 exists because I asked whether the system would actually score well if evaluated on DocSplit.** Measuring what an evaluator would see — rather than trusting our own benchmark script — showed the shipped configuration scored F1 0.110 with page grouping accuracy 0.216, and that the 0.823 previously reported had been obtained at a threshold refitted on the benchmark. That produced the calibration-driven expected-count rule now shipped, which lifted grouping accuracy to 0.615.
- **I questioned the choice of learner**, which had been inherited rather than justified. Benchmarking scikit-learn against XGBoost, LightGBM and CatBoost showed all four within 0.005 F1 — confirming the Stage 1 ceiling is representational — and exposed `xgboost` as a declared-but-unused dependency.
- **I asked whether other public datasets could improve Stage 1**, which led to evaluating against the DocSplit benchmark and to adding lift-over-trivial, the metric that revealed our best-looking score was the do-nothing baseline.
- **When an AI-generated code review diagnosed the Stage 2 table failure as a propagation bug, I required the diagnosis to be verified before it was implemented.** It was wrong: propagation worked, and the failure was in detection, upstream. Implementing that diagnosis as written would have rewritten correct code and left the real defects in place.

Approaches the tool proposed were rejected where measurement did not support them — including a transformer encoder measured at +0.115 R@1 that was not adopted because it could not be installed reliably, and five boundary-detection experiments that failed and are reported as negative results.

All numbers in this report were produced by executing the scripts in `scripts/`. None were generated by a model, and the repository's commit history reflects the same tooling declared here.

Reproducing every number

Full command sequence in README.md. In short: fetch TABME++ (train + val) for the shipped English boundary model, OpenPSS (train + full test) for the Dutch alternative, and RVL-CDIP text for the classifier; train the models; generate the two annotated fixtures plus the stress packet (`generate_stress_packet.py`); then run `evaluate_openpss_boundary.py` against the TABME++, OpenPSS and DocSplit manifests, followed by `evaluate_stage2.py` and `evaluate_stage3.py`. Every table above is written to `outputs/benchmarks/`.

The negative results reproduce via `--no-synthetic-blocks`, `--class-weight none`, and `train_rvlcdip_boundary.py`. The cosine-only ablation behind §6.1 reproduces via `ablate_cosine_boundary.py`, and the language finding in §5.1 by training the same architecture on both `data/raw/tabme/manifest.json` and `data/raw/openpss/train/manifest.json` and evaluating each on the other's held-out split. `pytest tests/ -q` runs 43 tests.